p0476r2
Bit-casting object representations

Published Proposal, 10 November 2017

This version:
http://wg21.link/P0476r2

Author:
JF Bastien (Apple)

Audience:
LWG

Project:
ISO JTC1/SC22/WG21: Programming Language C++

Source:
github.com/jfbastien/papers/blob/master/source/P0476r2.bs

Implementation:
github.com/jfbastien/bit_cast/

Abstract

Obtaining equivalent object representations The Right Way™.

Table of Contents

This paper is a revision of [P0476r1], addressing LEWG comments from the 2017 Toronto meeting as well as comments from LEWG and LWG from the 2017 Albuquerque meeting. See §4 Revision History for details.

§ 1. Background

Low-level code often seeks to interpret objects of one type as another: keep the same bits, but obtain an object of a different type. Doing so correctly is error-prone: using `reinterpret_cast` or `union` runs afoul

of type-aliasing rules yet these are the intuitive solutions developers mistakenly turn to.

Attuned developers use `aligned_storage` with `memcpy`, avoiding alignment pitfalls and allowing them to bit-cast non-default-constructible types.

This proposal uses appropriate concepts to prevent misuse. As the sample implementation demonstrates we could as well use `static_assert` or template SFINAE, but the timing of this library feature will likely coincide with concept's standardization.

Furthermore, it is currently impossible to implement a `constexpr` bit-cast function, as `memcpy` itself isn't `constexpr`. Marking the proposed function as `constexpr` doesn't require or prevent `memcpy` from becoming `constexpr`, but requires compiler support. This leaves implementations free to use their own internal solution (e.g. LLVM has a `bitcast` opcode).

We should standardize this oft-used idiom, and avoid the pitfalls once and for all.

§ 2. Proposed Wording

Below, substitute the `�` character with a number or name the editor finds appropriate for the sub-section.

In 20.5.1.2 **[headers]** add the header `<bit>` to:

- Table 16 — C++ library headers
- Table 19 — C++ headers for freestanding implementations

In the numerics section, add the following:

§ 2.1. 29. Bit manipulation library [bit]

§ 2.2. 29..1 General [bit.general]

The header `<bit>` provides components to access, manipulate and process both individual bits and bit sequences.

§ 2.3. 29..2 Header `<bit>` synopsis [bit.syn]

```
namespace std {
  // 29..3 bit_cast
  template<typename To, typename From>
  constexpr To bit_cast(const From& from) noexcept;
}
```

§ 2.4. 29..3 Function template `bit_cast` [bit.cast]

```
template<typename To, typename From>
constexpr To bit_cast(const From& from) noexcept;
```

1. *Remarks:*

   This function shall not participate in overload resolution unless:

   - `sizeof(To) == sizeof(From)` is `true`;
   - `is_trivially_copyable_v<To>` is `true`; and

- `is_trivially_copyable_v<From>` is `true`.

This function shall be `constexpr` if and only if `To`, `From`, and the types of all subobjects of `To` and `From` are types `T` such that:

- `is_union_v<T>` is `false`;
- `is_pointer_v<T>` is `false`;
- `is_member_pointer_v<T>` is `false`;
- `is_volatile_v<T>` is `false`; and
- `T` has no non-static data members of reference type.

2. *Returns*:

An object of type `To`. Each bit of the value representation of the result is equal to the corresponding bit in the object representation of `from`. Padding bits of the `To` object are unspecified. If there is no value of type `To` corresponding to the value representation produced, the behavior is undefined. If there are multiple such values, which value is produced is unspecified.

2.5. Feature testing

The `__cpp_lib_bit_cast` feature test macro should be added.

3. Appendix

The Standard's [**basic.types**] section explicitly blesses `memcpy`:

> For any trivially copyable type `T`, if two pointers to `T` point to distinct `T` objects `obj1` and `obj2`, where neither `obj1` nor `obj2` is a base-class subobject, if the *underlying bytes* (1.7) making up `obj1` are copied into `obj2`, `obj2` shall subsequently hold the same value as `obj1`.
>
> [*Example:*
>
> ```
> T* t1p;
> T* t2p;
> // provided that t2p points to an initialized object ...
> std::memcpy(t1p, t2p, sizeof(T));
> // at this point, every subobject of trivially copyable type in *t1p contains
> // the same value as the corresponding subobject in *t2p
> ```
>
> — *end example*]

Whereas section [**class.union**] says:

> In a union, at most one of the non-static data members can be active at any time, that is, the value of at most one of the non-static data members can be stored in a union at any time.

4. Revision History

4.1. r1 ➡ r2

The paper was reviewed by LEWG at the 2017 Toronto meeting and feedback was provided. In the 2017 Albuquerque meeting LEWG provided feedback regarding usage of concepts while discussing [P0802r0], and EWG reviewed the paper:

- Use "shall not participate in overload resolution" wording instead of a requires clause.
- The author was asked to explore naming. LEWG took a poll in Albuquerque and voted to keep `bit_cast`.

- There was strong sentiment that this facility should be available in freestanding implementations. LEWG is changing its guidance regarding freestanding header granularity, but until guidance is actually changed it was decided that a currently freestanding header should be used. LEWG took a poll in Albuquerque, and the new `<bit>` header was chosen instead of `<cstddef>`.
- Call out that `constexpr` requires compiler support.
- Make `constexpr` conditional, similar to variant's [variant.ctor] wording, based on an EWG straw poll in Albuquerque.
- LWG review made the `constexpr` remark recursive, and tuned the return wording, asking CWG to review the changes.
- LWG review requested that this paper also add the `<bit>` header, and let the editor resolve races if multiple papers add the header concurrently.
- CWG substantially tuned the wording.

4.2. r0 ➡ r1

The paper was reviewed by LEWG at the 2016 Issaquah meeting:

- Remove the standard layout requirement—trivially copyable suffices for the `memcpy` requirement.
- We discussed removing `constexpr`, but there was no consent either way. There was some suggestion that it'll be hard for implementers, but there's also some desire (by the same implementers) to have those features available in order to support things like `constexpr` instances of `std::variant`.
- The pointer-forbidding logic was removed. It was initially there to help developers when a better tool is available, but it's easily worked around (e.g. with a `struct` containing a pointer). Note that this doesn't prevent `constexpr` versions of `bit_cast`: the implementation is allowed to error out on `bit_cast` of pointer.
- Some discussion about concepts-usage, but it seems like mostly an LWG issue and we're reasonably sure that concepts will land before this or in a compatible vehicle.

Straw polls:

- Do we want to see [P0476r0] again? unanimous consent.
- `bit_cast` should allow pointer types in `To` and `From`. **SF F N A SA** 4 5 4 2 1
- `bit_cast` should be `constexpr`? **SF F N A SA** 4 3 7 2 3

5. Acknowledgement

Thanks to Saam Barati, Jeffrey Yasskin, and Sam Benzaquen for their early review and suggested improvements.

References

Informative References

[P0476r0]
JF Bastien. Bit-casting object representations. 16 October 2016. URL: https://wg21.link/p0476r0

[P0476r1]
JF Bastien. Bit-casting object representations. 11 November 2016. URL: https://wg21.link/p0476r1

[P0802r0]
Beman Dawes, Nicolai Josuttis, Walter E. Brown, Bob Steagall. Applying Concepts to the Standard Library. URL: https://wg21.link/p0802r0